

## OpenGL – Rendu de scène avancé et instancing

[Malek.Bengougam@gmail.com](mailto:Malek.Bengougam@gmail.com)

Les GPUs ont un fonctionnement hautement parallèle ce qui permet un fonctionnement asynchrone, par exemple l'exécution des appels de rendu (*draw call*) – ce qui correspond en OpenGL à chaque appel d'une fonction type ***glDrawArrays()*** etc... Cependant comme ces appels sont exécutés de façon séquentielle lors d'étapes du Pipeline l'exécution finale doit rester séquentielle (garantir l'ordre des appels de rendu). Les fonctions de rendu sont donc très largement bloquantes bien que le pilote essaye de masquer ces latences de différentes manières.

Note : en dehors des fonctions de rendu, d'autres fonctions provoquent également des blocages – que l'on nomme points de synchronisation. Il s'agit le plus souvent des fonctions de copie/mise-à-jour/transfert de données (***glBufferData()***, ***glTexImage2D()***, ***glUniform\*\*()***, ***glMapBuffer()*** etc ..., à noter cependant que ***glMapBufferRange()*** offre l'option de désactiver toute synchronisation au risque d'obtenir un rendu incohérent si l'on n'y prend pas garde.

L'Instancing -ou Geometry Instancing, qu'il soit software ou hardware, concerne le plus souvent le rendu de multiples maillages identiques, et désigne un ensemble de techniques permettant de réduire le nombre de « draw calls ».

Cette technique se rapproche du concept de rendu par lots (*batch rendering*) qui consiste à rendre un objet ou une scène en un minimum d'appel.

Le rendu par lots consiste à grouper les primitives, maillages etc... en fonction des états de rendu commun (même shaders, même matériaux, même textures, mêmes paramètres uniformes etc...).

Le but est ici de réduire le nombre d'appels de rendu OpenGL (***glDraw\*\*\*()***) ou de changement d'états (***glEnable()***, ***glCullFace()***, ***glDepthMask()***, ***glUniform\*\*\*()*** etc...)

### Technique de rendu naïve

For each Objet in scene

For each Model in Objet

Set matrices, constantes etc.: ***glUniform\*\*\*()*** // on a vu qu'un UBO pouvait aider ici

For each Mesh in Model

***glUseProgram()***

***glEnable()*** et autres changements d'états

***glBindVertexArrays()***

***glDraw\*\*\*()***

Généralement un logiciel de modélisation (ou un outil interne) va séparer un modèle 3D en plusieurs maillages (ou parties) lorsque ces derniers utilisent des matériaux différents.

On peut séparer le cas des objets identiques (même VAO, et donc VBO, même matériaux, et donc shaders etc...) du cas générique. Dans tous les cas, on aura des objets positionnés à des endroits différents de la scène ce qui signifie une matrice de transformation objet->monde (Model/World Matrix) spécifique à chaque objet ou instance.

## Technique 1 : regroupement de géométries identiques (software)

La première technique consiste à grouper plusieurs instances d'un « mesh » dans un seul et unique VBO/IBO (et donc un seul et même VAO).

Il s'agit donc de pré-transformer les vertices de chaque instance par leur matrice de transformation et de stocker ces vertices dans un VBO (potentiellement très large). L'IBO doit être patché afin que les indices reflètent la position du vertex dans le VBO.

Cette technique est surtout intéressante si une grande majorité des objets sont similaires et sont visibles à l'écran (particules, herbes, astéroïdes ...) cependant elle nécessite une modification temps-réel constante et un transfert important de données.

Avantage :

- Facile à mettre en oeuvre

Désavantage :

- coût CPU potentiellement important
- Impact important en termes de bande passante
- duplication des sommets, coût mémoire assez élevé

## Technique 2a : regroupement de géométries différentes (software)

On peut étendre la technique précédente à des maillages multiples, chacun possédant éventuellement des indices qui lui sont propres.

Dans un premier temps on groupe les instances d'un même mesh ensemble – ce qui revient au même processus que lors de la technique précédente, puis on concatène dans le VBO et l'IBO chacun de ces groupes.

On peut alors utiliser la fonction *glMultiDrawElements()* ou *glMultiDrawArrays()* qui fonctionne comme une boucle 'for' parcourant plusieurs tableaux d'indices.

Avantage :

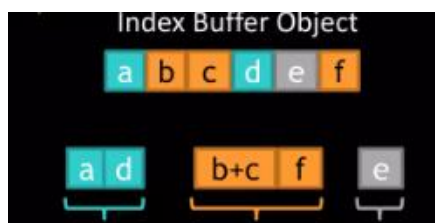
- support natif au niveau du pilote (software)

Désavantage :

- Même désavantages que la technique 1
- pas supporté sur mobile
- Pas possible tel quel de gérer des matériaux différents

## Technique 2b : regroupement de géométries en lots (software)

Afin de contourner la problématique des matériaux différents on va s'autoriser à faire plus d'appels de rendu en groupant en lots (batch). Cette technique reprend les bases de la technique 1b mais on constitue maintenant des groupes de rangs d'indices pour les objets ayant les mêmes propriétés. Malheureusement ce n'est pas possible sur mobile.



En effet, les paramètres de *glMultiDrawElements()* sont des tableaux d'entiers : le premier tableau contient le nombre d'indices constituant chaque mesh, et le second tableau contient des offsets (premier élément d'un mesh) dans l'IBO.

Illustration: 3 batches (3 appels à *glMultiDrawElements()*) pour lesquels chaque groupe de meshes [a,d], [b,c,f], [e] utilisent des propriétés différentes (shaders, uniforms...). Notez que comme b et c sont identiques et contigus dans l'IBO on peut les considérer comme un seul objet. Idéalement il faudrait grouper ainsi.

## Technique 3 : pseudo hardware instancing

a. On peut transposer la technique 1 dans le Vertex Shader en passant les matrices comme attribut de vertex. D'une manière naïve on pourrait ajouter une matrice à chaque sommet ce qui aurait pour résultat d'augmenter la taille mémoire requise.

*Note: Un appel à **glVertexAttribPointer**(N, ) ne peut contenir que 4 composantes. Pour passer une matrice 4x4 il faut donc faire 4 appels à **glVertexAttribPointer**(N, N+1, N+2, N+3 et de même pour **glEnableVertexAttribArray**).*

b. Une alternative moins gourmande consiste à transmettre ces données sous forme d'un tableau de matrices, et de n'ajouter qu'un simple indice à chaque sommet permettant ainsi d'indexer le tableau de matrices.

*Note: il faut utiliser **glVertexAttribIPointer**() -remarquez le 'I' majuscule entre 'Attrib' et 'Pointer' afin de passer des données de types entières.*

Deux possibilités s'offrent à nous en VRAM : les variables uniformes et les textures. Pour ce qui concerne les variables uniformes (en mémoire locale du GPU) il y'a une limitation quant au nombre de vecteurs (ou *locations*) disponibles. Cela reste possible mais il faudrait *splitter* le batch en plusieurs parties (plusieurs *draw calls*).

D'un autre côté, les textures (en mémoire globale, VRAM) ont plus de latence mais offrent des limites moins restrictives. Les accès aléatoires risquent d'être plus lent mais au moins il n'y'aura pas besoin de splitter le batch de rendu.

En revanche, les textures sont adressées via des coordonnées normalisées. Donc si l'on choisit de stocker le tableau de matrices sous forme de texture il faudra donc convertir les indices en coordonnées 2D en normalisé.

Il existe une alternative : utiliser **texelFetch**() plutôt que **texture**() dans le shader -e qui permet d'utiliser des coordonnées entières non normalisées.

Cette technique est particulièrement utile sur des hardware limités ou ne supportant pas l'*instancing* (du fait des spécifications, ex : WebGL et OpenGL ES 2.0, ou du fait des limites du matériel).

c. En OpenGL 3+ on dispose d'un nouveau type de buffer : les **Texture Buffer Objects (TBO)**. Ils permettent d'utiliser une texture comme s'il s'agissait d'un tableau d'uniformes (ou d'un UBO) tout en supportant l'usage de **texelFetch**().

Avantage :

- support hardware
- adaptable à plusieurs plateformes ne supportant pas l'instancing hardware

Désavantage :

- Un peu complexe à mettre en œuvre (bien faire attention aux offsets)
- Nécessite OpenGL 3 ou des extensions pour *texelFetch* et/ou les *TBO*

## Technique 4 : hardware instancing

Avec OpenGL 3.+ on dispose de la variable prédéfinie **gl\_InstanceID** qui est un identifiant de l'instance actuellement rendue.

Cette variable, à l'instar de **gl\_VertexID** qui est incrémentée à chaque traitement d'un sommet, est, elle, incrémentée chaque fois qu'une instance est traitée par OpenGL.

Concrètement on utilise les variantes des fonctions de rendu **glDrawArraysInstanced()** et **glDrawElementsInstanced()** qui prennent un paramètre supplémentaire : le nombre total d'instances à dessiner.

Maintenant que l'on peut facilement identifier une instance spécifique, on peut passer les données sous la forme d'attributs (comme dans la technique 3.b) ou bien, à l'instar de la technique 3.c, passer des données arbitraires mais cette fois en utilisant pleinement OpenGL 3.+ via des UBO (qui restent cependant limités en taille).

Si le hardware supporte OpenGL 4.+ il est également possible d'utiliser des "buffers" génériques. Le type du buffer à générer est **GL\_SHADER\_STORAGE\_BUFFER** (SSBO) mais en pratique on peut "bind" un VBO, etc. C'est sans doute la meilleure façon de procéder pour peu que l'on n'ait pas atteint les limites hardware.

## Technique 5 : hardware instancing (extension ARB\_instanced\_arrays)

Une autre alternative consiste à passer les data d'instance en tant qu'attributs.

Comme on le sait, si on utilise **glVertexAttribPointer()** de manière classique il faut alors dupliquer les valeurs pour chaque sommets ! Ce qui prends de la place en VRAM.

C'est là que la fonction **glVertexAttribDivisor()** entre en jeu. Cette fonction permet de spécifier la fréquence de transfert des données pour chaque attribut.

Par défaut cette valeur vaut zéro, ce qui indique au hardware que les attributs passés aux shaders doivent être lu / mis à jour à chaque sommet.

Une valeur supérieure ou égale à 1 permet d'indiquer que les attributs concernés ne seront lus qu'une fois par instance (si le Divisor vaut 1), ou une fois toutes les deux instance (si le Divisor vaut 2) etc...

Comme on l'a vu, on ne peut passer plus d'un **vec4** à la fois : dans le cas d'une matrice 4x4 il faudrait utiliser 4 canaux d'attributs.

**Exercices** : programmez un rendu simple d'instance où vous passerez les paramètres comme la matrice World ou encore des teintes de couleur, des identifiant de texture etc... en utilisant les techniques 4 et/ou 5.

**optionnel**: plutôt que de passer l'ensemble de la matrice, n'envoyer que des paramètres type « transform » (position, angles d'euler etc...) et reconstituez la matrice (ou les calculs de transformation) dans le Vertex Shader, par ex simuler les quaternions en GPU

## Technique 6 : regroupement de géométries différentes (hardware)

Nous avons vu qu'il était possible d'optimiser le rendu de meshes différents en les groupant par lots en fonction de critères de rendu (matériaux, etc...).

Avec OpenGL 4+ on dispose de la variable prédéfinie **gl\_DrawID** qui permet d'identifier un appel de rendu spécifique lorsque l'on utilise **glMultiDrawElements()**.

Dès lors, il est possible de passer des tableaux contenant des données spécifiques à chaque batches en indexant le tableau à l'aide de **gl\_DrawID**.

Reste qu'il n'est pas pratique de devoir patcher les indices dans l'IBO à chaque fois que l'on concatène un maillage dans un VBO. Le coût CPU peut devenir prohibitif.

OpenGL propose la fonction **glDrawElementsBaseVertex()** qui permet de spécifier la valeur de base que l'on doit ajouter à chaque indice.

Supposons deux maillages dans un VBO : le maillage A est stocké dans la plage de vertices [0; N-1] et le maillage B dans la plage de vertices [N; M-1].

Comme les vertices du second maillage commencent à l'indice N, il faudrait ajouter N à tous les indices de l'IBO référant au maillage B.

Avec **glDrawElementsBaseVertex(..., N)** le GPU ajoutera automatiquement N aux indices du maillage B. Et il est possible de faire de même avec plusieurs batches en utilisant **glMultiDrawElementsBaseVertex(..., GLint baseVertices[])** où **baseVertices[]** est un tableau d'entiers indiquant le décalage de base des indices pour chaque plage d'indices.

Cf [https://www.khronos.org/opengl/wiki/Vertex\\_Rendering#Base\\_Index](https://www.khronos.org/opengl/wiki/Vertex_Rendering#Base_Index)

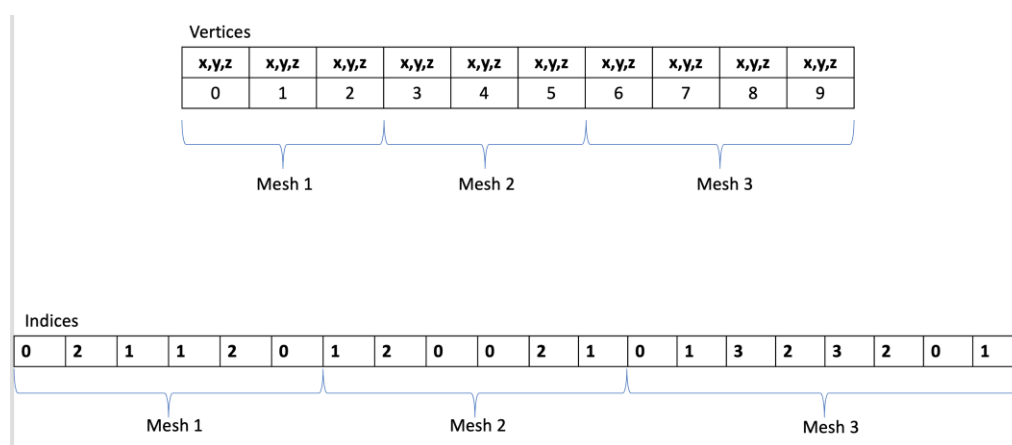


Illustration: avec **gl\*\*\*BaseVertex()** on peut conserver les indices originaux des maillages

Ces fonctions ne sont disponibles qu'à partir d'OpenGL 3.x, qui propose également **glDrawElementsInstancedBaseVertex()**. Cette fonction permet d'utiliser l'instancing sur un maillage stocké dans un large VBO/IBO. On n'a donc plus besoin de différencier meshes instanciés et meshes non-instanciés et on peut les regrouper en un VAO.

Cependant, cette technique n'est plus compatible avec l'utilisation de l'instancing d'attributs via **glVertexAttribDivisor()**. Ceci parce que le hardware n'utilise pas le numéro du vertex dans le calcul mais le numéro de l'instance qui ne varie pas ici.

C'est pourquoi OpenGL 4.2+ dispose de **glDrawElementsInstancedBaseInstance()** ainsi que de **glDrawElementsInstancedBaseVertexBaseInstance()** ! On peut donc combiner groupage dans un IBO/VBO/VAO et groupage des données d'instance.

**Quelques limites tout de même**

Tout ce qui a été dit concernant BaseVertex et BaseInstance n'est vraiment utile qu'à partir du moment où les meshes partagent les mêmes attributs (position, normale etc...) car un VAO fait le lien entre la signature du shader (les attributs en entrée, l'input layout dans le jargon), et le VBO.

Certaines fonctions ne sont disponibles qu'en OpenGL 4.2 et au-delà, et d'autres comme **glMultiDrawElements()** ne sont pas disponibles en OpenGL ES et en WebGL, on n'a donc pas accès non plus à `gl_DrawID` en dehors d'OpenGL 4.

Il existe des alternatives mais elles nécessitent en général un support d'OpenGL 4.2 voire 4.3 et supérieur.

On peut par exemple découpler l'input layout (signature du shader) du format du VBO (stream), et utiliser des fonctions indépendantes du VAO pour changer cela en temps réel.

On pourrait même se passer totalement de VAO (et VBO voire des IBO) en utilisant les buffers génériques (SSBO) : on bind nos tableaux (VBO voire IBO) en tant que `GL_SHADER_STORAGE_BUFFER` et dans le Vertex Shader on utilise `gl_VertexID` afin de lire le buffer comme s'il s'agissait d'un tableau de sommet. Ce n'est pas forcément plus optimal (voire sous-optimal si les données sont mal agencées en mémoire) mais le gain en souplesse de travail (et le fait qu'on n'ait plus besoin de VAO) est appréciable.

A défaut de pouvoir faire ces changements ou de disposer d'OpenGL 4.2+ on peut tout de même améliorer les performances d'une scène de plusieurs façons :

### Limiter le nombre de changements d'états

On peut définir une heuristique simple visant à réduire les changements d'états par image. Il peut être intéressant de grouper les changements d'états par fréquence.

Par exemple, les matrices de vue et de projection ne changent que si l'utilisateur se déplace ou modifie les paramètres de projection (fov, aspect ratio...) et ces données sont communes à l'ensemble des objets de la scène.

A l'opposé, on peut être amené à changer une variable uniform entre chaque draw call.

On peut définir un certain nombre de règles :

- N'utilisez **glUniform\*\*** que peu de fois, et pour des petites données ( $\leq \text{vec4}$ )
- Sinon regroupez les uniform dans un UBO, sous forme d'un tableau.
- Groupez les données par fréquence :
  - o Un UBO pour les données communes
  - o Un UBO pour les données spécifiques à la scène (lumières, etc...)
  - o Un UBO pour les données spécifiques au batch (matériaux...)
  - o Un UBO ou instancing pour les données spécifiques au mesh

Note: Il est fort probable qu'un UBO ne puisse stocker l'ensemble des données nécessaires (64 KiB grand max en général). Dans ce cas les TBO ou mieux, les SSBO, peuvent les remplacer avec plus de souplesse moyennant une légère perte de performance.

Une autre problématique est celle des textures. On peut regrouper les batches en combinant les textures dans une plus grande texture (un atlas ou sheet). Si les textures sont de même dimensions, OpenGL 3.+ dispose également des `TEXTURE_ARRAY`.